

```
#!/usr/bin/env python3
"""TTW Crypto Scanner V1.
```

Scans the top 25 non-stable cryptocurrencies on Binance Spot USDT pairs for three-wick trendline setups and sends live alerts to Telegram.

V1 strategy rules implemented:

- Three consecutive candles.
- Bullish setup uses candle lows; bearish setup uses candle highs.
- Trendline is drawn from wick 1 to the LIVE wick 3.
- Wick 2 may miss the line within a configurable tolerance (default 0.5%).
- Trendline may rise, fall, or be flat.
- Candle colour is irrelevant.
- Candle 3 is still forming; alert is sent before it closes.
- Entry reference = candle 3 open.
- SL1 = just beyond wick 3; SL2 = just beyond wick 2.
- One alert per symbol/timeframe/direction/current candle.
- Telegram / buttons record feedback.

This bot DOES NOT place trades.

```
"""
```

```
from __future__ import annotations
```

```
import asyncio
```

```
import csv
```

```
import hashlib
```

```
import json
```

```
import logging
```

```
import math
```

```
import os
```

```
import signal
```

```
import time
```

```
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Sequence, Tuple
```

```
import urllib.error
import urllib.parse
import urllib.request
```

```
BINANCE_API = os.getenv("BINANCE_API_BASE", "https://api.binance.com").rstrip("/")
COINGECKO_API = os.getenv("COINGECKO_API_BASE", "https://api.coingecko.com/api/v3").rstrip("/")
TELEGRAM_API = "https://api.telegram.org"
```

```
TIMEFRAMES: Dict[str, Tuple[str, int, str]] = {
    "2H": ("2h", 1, "native"),
    "3H": ("1h", 3, "hour"),
    "4H": ("4h", 1, "native"),
    "6H": ("6h", 1, "native"),
    "12H": ("12h", 1, "native"),
    "1D": ("1d", 1, "native"),
    "2D": ("1d", 2, "day"),
    "3D": ("3d", 1, "native"),
    "5D": ("1d", 5, "day"),
    "1W": ("1w", 1, "native"),
    "2W": ("1w", 2, "week"),
    "1M": ("1M", 1, "native"),
    "2M": ("1M", 2, "month"),
    "3M": ("1M", 3, "month"),
}
```

```
BASE_LIMITS = {
```

```
"1h": 16,  
"2h": 5,  
"4h": 5,  
"6h": 5,  
"12h": 5,  
"1d": 28,  
"3d": 5,  
"1w": 9,  
"1M": 13,  
}
```

```
EXCLUDED_BASES = {  
    # USD/stable-style assets  
    "USDT", "USDC", "FDUSD", "TUSD", "DAI", "USDE", "USDS", "PYUSD", "USD1",  
    "BUSD", "USDP", "GUSD", "FRAX", "LUSD", "SUSD", "EUR", "EURC", "EURI",  
    # Wrapped / liquid-staking duplicates that commonly crowd market-cap lists  
  
    "WBTC", "WETH", "STETH", "WSTETH", "WEETH", "WBETH", "RETH", "CBETH",  
}
```

```
LEVERAGED_SUFFIXES = ("UP", "DOWN", "BULL", "BEAR")  
STATE_PATH = Path(os.getenv("STATE_PATH", "state.json"))  
FEEDBACK_PATH = Path(os.getenv("FEEDBACK_PATH", "feedback.csv"))
```

```
@dataclass(frozen=True)  
class Candle:  
    open_time: int  
    open: float  
    high: float  
    low: float  
    close: float  
    volume: float
```

close_time: int

@dataclass(frozen=True)

class Setup:

direction: str # BULLISH / BEARISH

symbol: str

timeframe: str

entry: float

current_price: float

wick1: float

wick2: float

wick3: float

expected_wick2: float

wick2_deviation_pct: float

slope_pct_per_candle: float

candle3_open_time: int

candle3_close_time: int

sl1: float

sl2: float

class Config:

def __init__(self) -> None:

self.telegram_token = os.getenv("TELEGRAM_BOT_TOKEN", "").strip()

self.telegram_chat_id = os.getenv("TELEGRAM_CHAT_ID", "").strip() or None

e

self.scan_interval = max(10, int(os.getenv("SCAN_INTERVAL_SECONDS", "30")))

self.universe_refresh = max(900, int(os.getenv("UNIVERSE_REFRESH_SECONDS", "21600")))

self.top_n = min(50, max(1, int(os.getenv("TOP_N", "25"))))

self.wick2_tolerance_pct = max(0.0, float(os.getenv("WICK2_TOLERANCE

```

_PCT", "0.5"))))
    self.stop_buffer_pct = max(0.0, float(os.getenv("STOP_BUFFER_PCT", "0.10")))
    self.max_concurrency = min(30, max(2, int(os.getenv("MAX_CONCURRENCY", "12"))))
    self.symbol_override = [
        s.strip().upper().replace("/USDT", "").replace("USDT", "")
        for s in os.getenv("SYMBOLS", "").split(",")
        if s.strip()
    ]
    if not self.telegram_token:
        raise RuntimeError("TELEGRAM_BOT_TOKEN is required. Put the BotFather token in the environment.")

```

```

class TTWScanner:

```

```

    def __init__(self, config: Config) -> None:
        self.cfg = config
        self.stop_event = asyncio.Event()
        self.chat_id: Optional[str] = config.telegram_chat_id
        self.telegram_offset = 0
        self.alerted_keys = self._load_state()
        self.symbols: List[str] = []
        self.last_universe_refresh = 0.0
        self.semaphore = asyncio.Semaphore(self.cfg.max_concurrency)

```

```

    def _load_state(self) -> set[str]:
        try:
            if STATE_PATH.exists():
                data = json.loads(STATE_PATH.read_text())
                if isinstance(data, list):
                    return set(str(x) for x in data[-5000:])
        except Exception as exc:

```

```
logging.warning("Could not load state: %s", exc)
return set()
```

```
def _save_state(self) -> None:
    try:
        STATE_PATH.write_text(json.dumps(sorted(self.alerted_keys)[-5000:], i
ndent=2))
    except Exception as exc:
        logging.warning("Could not save state: %s", exc)
```

```
@staticmethod
def _sync_get_json(url: str, params: Optional[dict], timeout: int):
    if params:
        url = url + ("&" if "?" in url else "?") + urllib.parse.urlencode(params)
    req = urllib.request.Request(url, headers={"User-Agent": "TTW-Crypto-Sca
nner/1.0"})
    with urllib.request.urlopen(req, timeout=timeout) as r:
        return json.loads(r.read().decode("utf-8")), r.headers
```

```
@staticmethod
def _sync_post_json(url: str, payload: dict, timeout: int):
    body = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        url, data=body, method="POST",
        headers={"Content-Type": "application/json", "User-Agent": "TTW-Crypto
-Scanner/1.0"},
    )
    with urllib.request.urlopen(req, timeout=timeout) as r:
        return json.loads(r.read().decode("utf-8")), r.headers
```

```
async def _get_json(self, url: str, *, params: Optional[dict] = None, timeout: in
t = 15):
    async with self.semaphore:
```

```

for attempt in range(3):
    try:
        data, _headers = await asyncio.to_thread(self._sync_get_json, url, p
arams, timeout)
        return data
    except urllib.error.HTTPError as exc:
        if exc.code == 429 and attempt < 2:
            delay = float(exc.headers.get("Retry-After", "2"))
            await asyncio.sleep(max(1.0, delay))
            continue
        raise
    except (urllib.error.URLError, TimeoutError, json.JSONDecodeError):
        if attempt == 2:
            raise
            await asyncio.sleep(1.5 * (attempt + 1))
        raise RuntimeError(f"GET failed: {url}")

```

```

async def _post_json(self, url: str, payload: dict, timeout: int = 15):
    async with self.semaphore:
        for attempt in range(3):
            try:
                data, _headers = await asyncio.to_thread(self._sync_post_json, url,
payload, timeout)
                if not data.get("ok", True):
                    raise RuntimeError(f"Telegram/API error: {data}")
                return data
            except urllib.error.HTTPError as exc:
                if exc.code == 429 and attempt < 2:
                    delay = float(exc.headers.get("Retry-After", "2"))
                    await asyncio.sleep(max(1.0, delay))
                    continue
            try:
                detail = exc.read().decode("utf-8")

```

```
except Exception:
```

```
    detail = str(exc)
```

```
    raise RuntimeError(f"HTTP {exc.code}: {detail}") from exc
```

```
except (urllib.error.URLError, TimeoutError, json.JSONDecodeError, Ru  
ntimeError):
```

```
    if attempt == 2:
```

```
        raise
```

```
    await asyncio.sleep(1.5 * (attempt + 1))
```

```
async def telegram_call(self, method: str, payload: dict, timeout: int = 15):
```

```
    url = f"{TELEGRAM_API}/bot{self.cfg.telegram_token}/{method}"
```

```
    return await self._post_json(url, payload, timeout=timeout)
```

```
async def telegram_poll(self) -> None:
```

```
    """Poll Telegram for /start and feedback button presses."""
```

```
    while not self.stop_event.is_set():
```

```
        try:
```

```
            result = await self.telegram_call(
```

```
                "getUpdates",
```

```
                {"offset": self.telegram_offset, "timeout": 20, "allowed_updates": ["me  
ssage", "callback_query"]},
```

```
                timeout=25,
```

```
            )
```

```
            for update in result.get("result", []):
```

```
                self.telegram_offset = max(self.telegram_offset, int(update["update  
_id"]) + 1)
```

```
                msg = update.get("message")
```

```
                if msg:
```

```
                    text = (msg.get("text") or "").strip().lower()
```

```
                    candidate_chat = str(msg.get("chat", {}).get("id", ""))
```

```
                    if candidate_chat and (text.startswith("/start") or text.startswith("/  
status")):
```

```
                        if self.chat_id is None:
```

```

        self.chat_id = candidate_chat
        logging.info("Telegram chat auto-linked: %s", self.chat_id)
    if candidate_chat == self.chat_id:
        await self.telegram_call(
            "sendMessage",
            {
                "chat_id": self.chat_id,
                "text": self.status_text(),
                "disable_web_page_preview": True,
            },
        )
    cb = update.get("callback_query")
    if cb:
        await self.handle_callback(cb)
except Exception as exc:
    logging.warning("Telegram polling error: %s", exc)
    await asyncio.sleep(3)

```

def status_text(self) -> str:

```

    symbols = ", ".join(s.replace("USDT", "") for s in self.symbols[:10])
    more = "..." if len(self.symbols) > 10 else ""
    return (

```

```

        "✅ TTW Crypto Scanner V1 is online.\n\n"

```

```

        f"Universe: top {self.cfg.top_n} non-stable Binance USDT coins\n"

```

```

        f"Timeframes: {', '.join(TIMEFRAMES)}\n"

```

```

        f"Scan interval: {self.cfg.scan_interval}s\n"

```

```

        f"Wick-2 tolerance: {self.cfg.wick2_tolerance_pct:.2f}%\n"

```

```

        f"Current universe: {symbols}{more}\n\n"

```

```

        "It sends alerts only — it never places trades."

```

```

    )

```

async def handle_callback(self, cb: dict) -> None:

```

    data = cb.get("data") or ""

```

```

cb_id = cb.get("id")
if not data.startswith(("v|", "x|")):
    if cb_id:
        await self.telegram_call("answerCallbackQuery", {"callback_query_id":
cb_id})
    return
parts = data.split("|")
if len(parts) != 6:
    return
vote, symbol, timeframe, direction_code, candle_open, alert_id = parts
verdict = "VALID" if vote == "v" else "INVALID"
direction = "BULLISH" if direction_code == "B" else "BEARISH"
chat_id = str(cb.get("message", {}).get("chat", {}).get("id", ""))
self.write_feedback(alert_id, verdict, chat_id, symbol, timeframe, direction,
candle_open)
if cb_id:
    await self.telegram_call(
        "answerCallbackQuery",
        {"callback_query_id": cb_id, "text": f"Feedback recorded: {verdict.title()}
} "},
    )

```

```

def write_feedback(
    self,
    alert_id: str,
    verdict: str,
    chat_id: str,
    symbol: str,
    timeframe: str,
    direction: str,
    candle_open: str,
) -> None:
    exists = FEEDBACK_PATH.exists()

```

try:

```
with FEEDBACK_PATH.open("a", newline="") as f:
```

```
    writer = csv.writer(f)
```

```
    if not exists:
```

```
        writer.writerow([
```

```
            "recorded_utc", "alert_id", "verdict", "chat_id", "symbol",
```

```
            "timeframe", "direction", "candle3_open_time_ms",
```

```
        ])
```

```
    writer.writerow([
```

```
        datetime.now(timezone.utc).isoformat(), alert_id, verdict, chat_id,
```

```
        symbol, timeframe, direction, candle_open,
```

```
    ])
```

```
except Exception as exc:
```

```
    logging.warning("Could not write feedback: %s", exc)
```

```
async def refresh_universe(self) -> None:
```

```
    if self.cfg.symbol_override:
```

```
        self.symbols = [s + "USDT" for s in self.cfg.symbol_override]
```

```
        self.last_universe_refresh = time.time()
```

```
        return
```

```
exchange_info = await self._get_json(f"{BINANCE_API}/api/v3/exchangeInfo")
```

```
valid = set()
```

```
for item in exchange_info.get("symbols", []):
```

```
    symbol = item.get("symbol", "")
```

```
    base = item.get("baseAsset", "")
```

```
    if (
```

```
        item.get("status") == "TRADING"
```

```
        and item.get("quoteAsset") == "USDT"
```

```
        and item.get("isSpotTradingAllowed", True)
```

```
        and base not in EXCLUDED_BASES
```

```
        and not base.endswith(LEVERAGED_SUFFIXES)
```

```
);  
    valid.add(symbol)
```

```
selected: List[str] = []
```

```
try:
```

```
    markets = await self._get_json(  
        f"{COINGECKO_API}/coins/markets",  
        params={  
            "vs_currency": "usd", "order": "market_cap_desc", "per_page": 75,  
            "page": 1, "sparkline": "false",  
        },  
        timeout=20,  
    )
```

```
    for coin in markets:
```

```
        base = str(coin.get("symbol", "")).upper()
```

```
        pair = base + "USDT"
```

```
        if base in EXCLUDED_BASES or base.endswith(LEVERAGED_SUFFIXE
```

```
S):
```

```
            continue
```

```
        if pair in valid and pair not in selected:
```

```
            selected.append(pair)
```

```
            if len(selected) >= self.cfg.top_n:
```

```
                break
```

```
    except Exception as exc:
```

```
        logging.warning("CoinGecko market-cap universe failed; using Binance  
quote-volume fallback: %s", exc)
```

```
if len(selected) < self.cfg.top_n:
```

```
    tickers = await self._get_json(f"{BINANCE_API}/api/v3/ticker/24hr")
```

```
    ranked = []
```

```
    for t in tickers:
```

```
        symbol = t.get("symbol", "")
```

```
        if symbol not in valid or symbol in selected:
```

continue

try:

quote_volume = float(t.get("quoteVolume", 0.0))

except (TypeError, ValueError):

quote_volume = 0.0

ranked.append((quote_volume, symbol))

ranked.sort(reverse=True)

for _, symbol in ranked:

selected.append(symbol)

if len(selected) >= self.cfg.top_n:

break

self.symbols = selected[: self.cfg.top_n]

self.last_universe_refresh = time.time()

logging.info("Universe refreshed (%d): %s", len(self.symbols), ", ".join(self.symbols))

async def fetch_klines(self, symbol: str, interval: str) -> List[Candle]:

raw = await self._get_json(

f"{BINANCE_API}/api/v3/klines",

params={"symbol": symbol, "interval": interval, "limit": BASE_LIMITS[interval]}),

)

return [

Candle(

open_time=int(k[0]),

open=float(k[1]),

high=float(k[2]),

low=float(k[3]),

close=float(k[4]),

volume=float(k[5]),

close_time=int(k[6]),

)

```
    for k in raw
]
```

```
async def fetch_symbol_bases(self, symbol: str) -> Dict[str, List[Candle]]:
    intervals = sorted({v[0] for v in TIMEFRAMES.values()})
    tasks = [self.fetch_klines(symbol, interval) for interval in intervals]
    results = await asyncio.gather(*tasks, return_exceptions=True)
    out: Dict[str, List[Candle]] = {}
    for interval, result in zip(intervals, results):
        if isinstance(result, Exception):
            logging.warning("%s %s fetch failed: %s", symbol, interval, result)
        else:
            out[interval] = result
    return out
```

```
def candles_for_timeframe(self, base: Dict[str, List[Candle]], label: str) -> List[Candle]:
    interval, factor, mode = TIMEFRAMES[label]
    candles = base.get(interval, [])
    if factor == 1:
        return candles
    return aggregate_candles(candles, factor, mode)
```

```
async def scan_symbol(self, symbol: str) -> None:
    try:
        bases = await self.fetch_symbol_bases(symbol)
        now_ms = int(time.time() * 1000)
        for timeframe in TIMEFRAMES:
            candles = self.candles_for_timeframe(bases, timeframe)
            if len(candles) < 3:
                continue
            c1, c2, c3 = candles[-3:]
            # V1 only alerts while candle 3 is still live.
```

```
if not (c3.open_time <= now_ms <= c3.close_time + 2000):
```

```
    continue
```

```
for direction in ("BULLISH", "BEARISH"):
```

```
    setup = detect_setup(
```

```
        symbol=symbol,
```

```
        timeframe=timeframe,
```

```
        candles=(c1, c2, c3),
```

```
        direction=direction,
```

```
        tolerance_pct=self.cfg.wick2_tolerance_pct,
```

```
        stop_buffer_pct=self.cfg.stop_buffer_pct,
```

```
    )
```

```
    if setup is None:
```

```
        continue
```

```
    key = f"{symbol}|{timeframe}|{direction}|{c3.open_time}"
```

```
    if key in self.alerted_keys:
```

```
        continue
```

```
    if self.chat_id is None:
```

```
        logging.info("Setup found but Telegram chat is not linked yet: %s",
```

```
key)
```

```
        continue
```

```
    await self.send_alert(setup)
```

```
    self.alerted_keys.add(key)
```

```
    self._save_state()
```

```
except Exception as exc:
```

```
    logging.exception("Scan failed for %s: %s", symbol, exc)
```

```
async def send_alert(self, s: Setup) -> None:
```

```
    arrow = "●" if s.direction == "BULLISH" else "●"
```

```
    trend = "rising" if s.slope_pct_per_candle > 0.02 else "falling" if s.slope_pct_per_candle < -0.02 else "flat"
```

```
    sl1_pct = abs(s.entry - s.sl1) / s.entry * 100 if s.entry else 0
```

```
    sl2_pct = abs(s.entry - s.sl2) / s.entry * 100 if s.entry else 0
```

```
    opened = datetime.fromtimestamp(s.candle3_open_time / 1000, tz=tz=timezone.utc)
```

```

one.utc).strftime("%Y-%m-%d %H:%M UTC")
    closes = datetime.fromtimestamp(s.candle3_close_time / 1000, tz=timezo
ne.utc).strftime("%Y-%m-%d %H:%M UTC")
    alert_id = hashlib.sha1(
        f"{s.symbol}|{s.timeframe}|{s.direction}|{s.candle3_open_time}".encode(
    )
    ).hexdigest()[:8]
    dc = "B" if s.direction == "BULLISH" else "S"
    callback_tail = f"{s.symbol}|{s.timeframe}|{dc}|{s.candle3_open_time}|{aler
t_id}"
    chart_url = f"https://www.tradingview.com/chart/?symbol=BINANCE%3A{
s.symbol}"

    text = (
        f"{arrow} TTW ALERT — {s.direction}\n\n"
        f"Asset: {s.symbol[:4]}/USDT\n"
        f"Timeframe: {s.timeframe}\n"
        f"Third candle opened: {opened}\n"
        f"Third candle closes: {closes}\n\n"
        f"Entry reference (candle 3 open): {fmt_price(s.entry)}\n"
        f"Current price: {fmt_price(s.current_price)}\n"
        f"Wick 1: {fmt_price(s.wick1)}\n"
        f"Wick 2: {fmt_price(s.wick2)}\n"
        f"Live wick 3: {fmt_price(s.wick3)}\n"
        f"Wick-2 line deviation: {s.wick2_deviation_pct:.3f}%\n"
        f"Trendline: {trend} ({s.slope_pct_per_candle:+.3f}% / candle)\n\n"
        f"SL1 — beyond wick 3: {fmt_price(s.sl1)} ({sl1_pct:.2f}% from entry)\n"
        f"SL2 — beyond wick 2: {fmt_price(s.sl2)} ({sl2_pct:.2f}% from entry)\n\
n"
        f" ⚠️ Scanner alert only. Check the chart yourself before acting; no trade
has been placed."
    )
    keyboard = {

```

```

"inline_keyboard": [
    [
        {"text": "✅ Valid setup", "callback_data": "v|" + callback_tail},
        {"text": "❌ Not valid", "callback_data": "x|" + callback_tail},
    ],
    [{"text": "📈 Open TradingView", "url": chart_url}],
]
}
await self.telegram_call(
    "sendMessage",
    {
        "chat_id": self.chat_id,
        "text": text,
        "reply_markup": keyboard,
        "disable_web_page_preview": True,
    },
)
logging.info("Alert sent: %s %s %s", s.symbol, s.timeframe, s.direction)

```

```

async def scanner_loop(self) -> None:
    while not self.stop_event.is_set():
        started = time.time()
        try:
            if not self.symbols or time.time() - self.last_universe_refresh >= self.cfg.universe_refresh:
                await self.refresh_universe()
            # Keep per-symbol fanout moderate. Each symbol internally fetches the 9 unique base intervals concurrently.
            for chunk_start in range(0, len(self.symbols), 5):
                chunk = self.symbols[chunk_start:chunk_start + 5]
                await asyncio.gather(*(self.scan_symbol(s) for s in chunk))
        except Exception as exc:
            logging.exception("Scanner cycle failed: %s", exc)

```

```
elapsed = time.time() - started
```

```
sleep_for = max(1.0, self.cfg.scan_interval - elapsed)
```

```
try:
```

```
    await asyncio.wait_for(self.stop_event.wait(), timeout=sleep_for)
```

```
except asyncio.TimeoutError:
```

```
    pass
```

```
async def run(self) -> None:
```

```
    # Polling and webhooks cannot be used simultaneously. Clear any old web  
hook and verify the token.
```

```
    await self.telegram_call("deleteWebhook", {"drop_pending_updates": False}  
)
```

```
    me = await self.telegram_call("getMe", {})
```

```
    logging.info("Telegram bot authenticated as @%s", me.get("result", {}).get("  
username", "unknown"))
```

```
    await self.refresh_universe()
```

```
    logging.info("TTW scanner started. Telegram chat: %s", self.chat_id or "wai  
ting for /start")
```

```
    tasks = [
```

```
        asyncio.create_task(self.telegram_poll(), name="telegram-poll"),
```

```
        asyncio.create_task(self.scanner_loop(), name="scanner"),
```

```
    ]
```

```
    await self.stop_event.wait()
```

```
    for task in tasks:
```

```
        task.cancel()
```

```
    await asyncio.gather(*tasks, return_exceptions=True)
```

```
def fmt_price(x: float) -> str:
```

```
    if x >= 1000:
```

```
        return f"{x:,.2f}"
```

```
    if x >= 1:
```

```
        return f"{x:,.4f}"
```

```
if x >= 0.01:
    return f"{x:.6f}"
return f"{x:.9f}"
```

```
def detect_setup(
    *,
    symbol: str,
    timeframe: str,
    candles: Sequence[Candle],
    direction: str,
    tolerance_pct: float,
    stop_buffer_pct: float,
) -> Optional[Setup]:
    if len(candles) != 3:
        raise ValueError("detect_setup expects exactly three candles")
    c1, c2, c3 = candles
    if direction not in {"BULLISH", "BEARISH"}:
        raise ValueError("direction must be BULLISH or BEARISH")

    if direction == "BULLISH":
        w1, w2, w3 = c1.low, c2.low, c3.low
    else:
        w1, w2, w3 = c1.high, c2.high, c3.high

    dt_total = c3.open_time - c1.open_time
    if dt_total <= 0:
        return None
    fraction = (c2.open_time - c1.open_time) / dt_total
    expected2 = w1 + (w3 - w1) * fraction
    if not math.isfinite(expected2) or expected2 == 0:
        return None
    deviation_pct = abs(w2 - expected2) / abs(expected2) * 100
```

```
if deviation_pct > tolerance_pct:  
    return None
```

```
avg = (abs(w1) + abs(w3)) / 2 or 1.0
```

```
# Two candle-spacings from wick 1 to wick 3 for consecutive target candles.
```

```
slope_pct_per_candle = ((w3 - w1) / 2) / avg * 100
```

```
buffer = stop_buffer_pct / 100.0
```

```
if direction == "BULLISH":
```

```
    sl1 = w3 * (1 - buffer)
```

```
    sl2 = w2 * (1 - buffer)
```

```
else:
```

```
    sl1 = w3 * (1 + buffer)
```

```
    sl2 = w2 * (1 + buffer)
```

```
return Setup(  
    direction=direction,
```

```
    symbol=symbol,
```

```
    timeframe=timeframe,
```

```
    entry=c3.open,
```

```
    current_price=c3.close,
```

```
    wick1=w1,
```

```
    wick2=w2,
```

```
    wick3=w3,
```

```
    expected_wick2=expected2,
```

```
    wick2_deviation_pct=deviation_pct,
```

```
    slope_pct_per_candle=slope_pct_per_candle,
```

```
    candle3_open_time=c3.open_time,
```

```
    candle3_close_time=c3.close_time,
```

```
    sl1=sl1,
```

```
    sl2=sl2,
```

```
)
```

```
def _month_add(dt: datetime, months: int) -> datetime:
```

```
    idx = dt.year * 12 + (dt.month - 1) + months
```

```
    return datetime(idx // 12, idx % 12 + 1, 1, tzinfo=timezone.utc)
```

```
def bucket_bounds(open_time_ms: int, factor: int, mode: str) -> Tuple[int, int]:
```

```
    dt = datetime.fromtimestamp(open_time_ms / 1000, tz=timezone.utc)
```

```
    if mode == "hour":
```

```
        epoch_hours = int(open_time_ms // 3_600_000)
```

```
        start_hours = (epoch_hours // factor) * factor
```

```
        start_ms = start_hours * 3_600_000
```

```
        end_ms = start_ms + factor * 3_600_000
```

```
    elif mode == "day":
```

```
        epoch_day = datetime(1970, 1, 1, tzinfo=timezone.utc)
```

```
        day_start = datetime(dt.year, dt.month, dt.day, tzinfo=timezone.utc)
```

```
        days = (day_start - epoch_day).days
```

```
        start = epoch_day + timedelta(days=(days // factor) * factor)
```

```
        end = start + timedelta(days=factor)
```

```
        start_ms = int(start.timestamp() * 1000)
```

```
        end_ms = int(end.timestamp() * 1000)
```

```
    elif mode == "week":
```

```
        # Monday-aligned ISO-style weeks, grouped in factor-week blocks.
```

```
        monday = datetime(dt.year, dt.month, dt.day, tzinfo=timezone.utc) - timed  
elta(days=dt.weekday())
```

```
        anchor = datetime(1970, 1, 5, tzinfo=timezone.utc) # Monday
```

```
        weeks = (monday - anchor).days // 7
```

```
        start = anchor + timedelta(weeks=(weeks // factor) * factor)
```

```
        end = start + timedelta(weeks=factor)
```

```
        start_ms = int(start.timestamp() * 1000)
```

```
        end_ms = int(end.timestamp() * 1000)
```

```
    elif mode == "month":
```

```

month_idx = dt.year * 12 + (dt.month - 1)
grouped_idx = (month_idx // factor) * factor
start = datetime(grouped_idx // 12, grouped_idx % 12 + 1, 1, tzinfo=timezone.utc)
end = _month_add(start, factor)
start_ms = int(start.timestamp() * 1000)
end_ms = int(end.timestamp() * 1000)
else:
    raise ValueError(f"Unsupported aggregation mode: {mode}")
return start_ms, end_ms

```

```

def aggregate_candles(candles: Sequence[Candle], factor: int, mode: str) -> List[Candle]:

```

```

    if factor <= 1:
        return list(candles)
    groups: Dict[int, List[Candle]] = {}
    ends: Dict[int, int] = {}
    for c in candles:
        start, end = bucket_bounds(c.open_time, factor, mode)
        groups.setdefault(start, []).append(c)
        ends[start] = end

```

```

    out: List[Candle] = []
    for start in sorted(groups):
        items = sorted(groups[start], key=lambda x: x.open_time)
        out.append(
            Candle(
                open_time=start,
                open=items[0].open,
                high=max(x.high for x in items),
                low=min(x.low for x in items),
                close=items[-1].close,

```

```
        volume=sum(x.volume for x in items),
        close_time=ends[start] - 1,
    )
)
return out
```

```
def configure_logging() -> None:
    level = os.getenv("LOG_LEVEL", "INFO").upper()
    logging.basicConfig(
        level=getattr(logging, level, logging.INFO),
        format="%(asctime)s | %(levelname)s | %(message)s",
    )
```

```
async def amain() -> None:
    configure_logging()
    cfg = Config()
    scanner = TTWScanner(cfg)
    loop = asyncio.get_running_loop()
    for sig in (signal.SIGINT, signal.SIGTERM):
        try:
            loop.add_signal_handler(sig, scanner.stop_event.set)
        except NotImplementedError:
            pass
    await scanner.run()
```

```
if __name__ == "__main__":
    asyncio.run(amain())
```